Oumelbenine Tayeb
Report

Application Layer: Design Patterns, DNS, HTTP, and QUIC and Cryptography and Security

This report provides a detailed analysis of Lectures 23 and 24, which transition from raw socket programming to the design of real-world application-layer protocols and the cryptographic frameworks required to secure them.

Lecture 23 explores the "wildly different choices" designers make when building protocols atop the transport layer. It frames the application layer as the area of network design with the most freedom, moving beyond the mechanics of bits and packets to the logic of user interactions

1. Design Patterns and the "Axes" of Protocols
The lecture introduces a framework for understanding any new protocol by identifying its position along several design axes:

- State: HTTP is stateless by design, meaning each request stands alone. State is managed via cookies, where the client carries the state (e.g., a session ID) and presents it to the server, which then looks it up in a database. This is a critical structural decision that allows for massive scaling via load balancers.
- Encoding: Transitioning from text-based (ASCII) formats like HTTP/1.1, which are easy to debug, to binary formats like HTTP/2 and HTTP/3, which prioritize performance and parsing speed.
- Architecture: The contrast between Client-Server (asymmetric roles, e.g., DNS, HTTP) and Peer-to-Peer (symmetric roles, e.g., BitTorrent). BitTorrent is highlighted as a masterpiece of mechanism design, using rarest-first chunk selection and tit-for-tat choking to ensure that popularity increases a file's distribution capacity rather than overwhelming a single server.

2. DNS: The Internet’s Phone Book
Despite being decentralized, DNS is hierarchical, not P2P. It scales through aggressive caching and Time-to-Live (TTL) values. The resolution process is asymmetric: a "stub resolver" on a laptop speaks recursively to a resolver (like 1.1.1.1), which then iteratively walks the chain from Root servers to TLD servers (.edu) to Authoritative servers (bu.edu).

3. The Evolution of HTTP
The lecture narrates the "HTTP evolution" as a series of solutions to specific performance bottlenecks:

- HTTP/1.1: The baseline. It suffered from application-level head-of-line (HOL) blocking, where one slow response blocked all subsequent requests on a connection.
- HTTP/2: Introduced binary framing and multiplexing, allowing many independent streams to share one TCP connection, thus solving application-level HOL blocking. However, it remained vulnerable to TCP-layer HOL blocking—if one TCP segment is lost, the entire connection stalls.

- HTTP/3 (QUIC): Runs over UDP to move transport logic into user space. It solves TCP-layer HOL blocking by providing reliable delivery per stream. It also introduces connection migration (keeping a session alive when switching from WiFi to cellular) and 0-RTT resumption.

Lecture 24 Cryptography and Security : L23 ended by noting that most protocols run in cleartext by default; L24 provides the cryptographic tools to "fix" that.

1. The Threat Model and Primitives
The lecture defines a threat model involving α (sender), β (receiver), and τ (adversary). It maps specific security requirements to cryptographic primitives:

- Confidentiality: Addressed by Symmetric Cryptography (AES-GCM). AES is hardware-accelerated on modern CPUs, making encryption "effectively free".
- Integrity and Authentication: Addressed by Hash Functions (SHA-256) and Message Authentication Codes (MACs).
- Non-Repudiation: Specifically requires Digital Signatures (Asymmetric Cryptography), as a MAC (shared key) cannot prove to a third party which of the two key-holders created a message.

2. Asymmetric Cryptography and RSA
The lecture provides a deep dive into RSA math, derived from Euler's Theorem. RSA's security relies on the hardness of factoring large semiprime numbers. While 2048-bit RSA is currently secure, the lecture notes that Shor's algorithm on a quantum computer could factor it in polynomial time, driving interest in post-quantum cryptography.

3. Diffie Hellman and Forward Secrecy
A critical advancement in TLS 1.3 is the mandatory use of ephemeral Diffie Hellman (DH). Unlike the old RSA key-transport mode, DH allows two parties to agree on a secret key without ever transmitting it. Because these keys are ephemeral and discarded after a session, a compromise of a server's long-term private key tomorrow cannot be used to decrypt traffic recorded today a property known as Forward Secrecy.

4. Public Key Infrastructure (PKI) and TLS 1.3
To solve the "trust problem"knowing that a public key actually belongs to bank.com the web relies on certificates signed by Certificate Authorities (CAs). Browsers ship with a "root store" of trusted CA keys. Let's Encrypt is credited with moving the web from 30% to 95% HTTPS by making certificates free and automated.

The TLS 1.3 handshake is the culmination of these primitives, achieving a secure connection in just one round-trip:

1. ClientHello: Sends supported ciphers and an ephemeral DH share.
2. ServerHello: Sends its DH share, certificate chain, and a signature over the handshake transcript.
3. Keys Derivation: Both parties derive symmetric AES-GCM keys from the DH exchange

Conclusion:The two lectures close the course's "arc" by showing how QUIC integrates the TLS 1.3 handshake directly into the transport layer. This delivers on the promise of a "full stack," where every layer from Shannon’s information theory to modern web applications is encrypted by default in a single round-trip.